

A. Informasi & Ringkasan

1. Identitas

NIM	11S23043
Nama	Grace Evelin Siallagan
Kelas	12 IF 2
Judul Praktikum	Trees & Hash Table
Video Presentasi	https://youtu.be/w1fLmOa-zZA

2. Capaian & Ringkasan Pemahaman

Hasil Capaian setelah mengikuti praktikum:

- A (Penyelesaian): Selesai (1), Tidak Selesai (0).
- B (Pemahaman): Tidak Paham (1), Kurang Paham (2), Cukup Paham (3), Paham (4), Sangat Paham (5).

No	Indikator	A	B
1	Trees	1	3
2	Hash Table	1	3
3	Studi Kasus Binary Search Tree	1	3
4	Studi Kasus Hash Table	1	3
5	Menghitung Big O	1	3

Dalam pemahaman saya, studi kasus mengenai Binary Search Tree dan Hash Table memberikan pemahaman yang lebih dalam tentang bagaimana struktur data tersebut bekerja secara praktis. Dengan mengimplementasikan penghapusan node, perhitungan jumlah node dan daun dalam BST, saya semakin memahami bagaimana tree berevolusi setelah operasi-operasi dilakukan. Sementara pada Hash Table, saya belajar bagaimana memanfaatkan struktur ini untuk menyelesaikan masalah umum seperti pencarian duplikasi dan identifikasi karakter unik dalam string secara efisien. Dari kedua studi kasus ini, saya juga mendapatkan wawasan tentang kompleksitas algoritma, di mana Binary Search Tree memiliki kompleksitas waktu rata-rata $O(\log n)$ untuk pencarian, penyisipan, dan penghapusan, sedangkan Hash Table menawarkan waktu akses rata-rata $O(1)$, tergantung pada distribusi hash-nya. Praktikum ini memperkuat pemahaman saya terhadap efisiensi

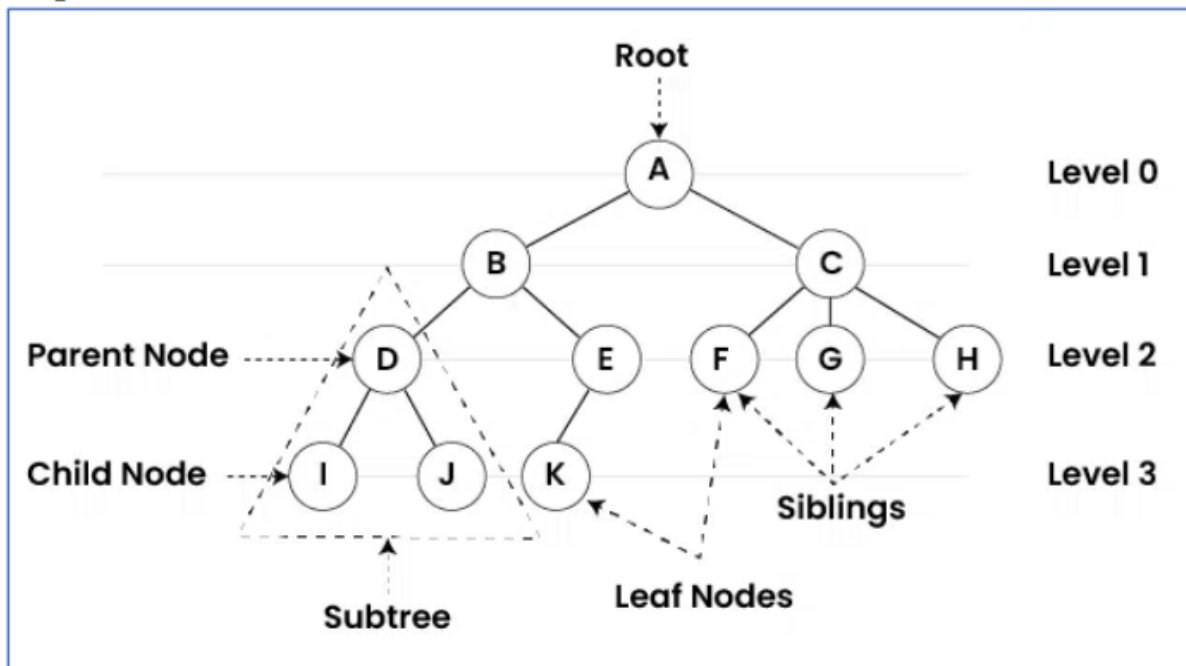
struktur data dan pentingnya memilih pendekatan yang tepat berdasarkan konteks permasalahan yang dihadapi.

B. Aktivitas Praktikum

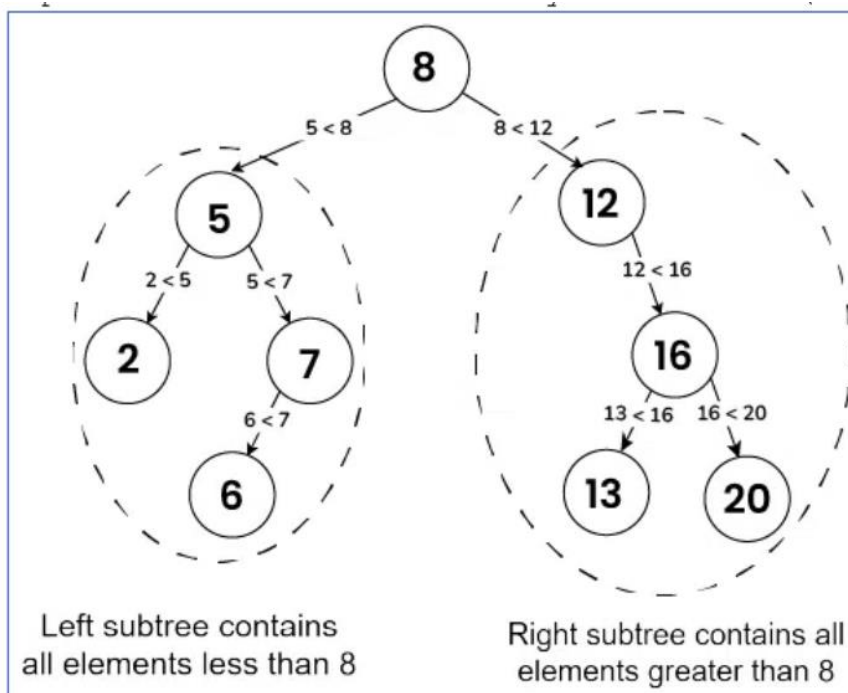
1. Trees

a). Intro

Representasi visual dari Trees:



Representasi visual dari Binary Search Tree (BST)



b) BST Constructor

Pada proyek Java dengan maven tambahkan file baru dengan nama "BinarySearchTree.java" dan "App.java".

```
1 package del.ifs23043;
2
3 public class BinarySearchTree {
4     public Node root;
5
6     class Node {
7         int value;
8         Node left;
9         Node right;
10
11         private Node(int value) {
12             this.value = value;
13         }
14     }
15
16     public Node getRoot() {
17         return root;
18     }
19 }
20
```

Kelas BinarySearchTree dalam kode ini merepresentasikan struktur data pohon biner pencarian (BST). Kelas ini memiliki atribut root sebagai node utama dari pohon. Di dalamnya terdapat kelas Node yang merepresentasikan setiap elemen dalam BST, yang terdiri dari atribut value untuk menyimpan nilai, serta left dan right sebagai referensi ke anak kiri dan kanan. Konstruktor Node bersifat privat, sehingga hanya dapat diakses dalam BinarySearchTree, membatasi pembuatan objek Node dari luar kelas ini. Selain itu, terdapat metode getRoot() yang mengembalikan node root dari BST. Namun, dalam implementasi ini belum terdapat metode untuk menambahkan, mencari, atau menghapus elemen dalam BST, sehingga fungsionalitasnya masih terbatas.

```
1 package del.ifs23043;
2
3 public class App {
4
5     public static void main(String[] args) {
6         BinarySearchTree myTree = new BinarySearchTree();
7         System.out.println("root = " + myTree.root);
8     }
9 }
10
```

Kode dalam kelas App berfungsi sebagai titik awal eksekusi program dengan membuat objek BinarySearchTree bernama myTree. Setelah objek dibuat, program mencetak nilai root dari myTree. Karena dalam kelas BinarySearchTree tidak ada inisialisasi atau metode untuk menambahkan elemen ke dalam pohon, nilai root secara default tetap null. Akibatnya, ketika program dijalankan, output yang dihasilkan adalah "root = null". Agar pohon biner pencarian dapat berfungsi dengan baik, diperlukan metode tambahan seperti insert() untuk menambahkan elemen dan mengisi nilai root sesuai aturan struktur data BST.

```
src > main > java > del > ifs23043 > J App.java > App
1 package del.ifs23043;
2 public class App {
3     public static void main(String[] args) {
4         BinarySearchTree myTree = new BinarySearchTree();
5         System.out.println("root = " + myTree.root);
6     }
7 }
8
```

```
ifs23043-alstrudat-p5> 6 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5\target\classes' 'del.ifs23043.App'
root = null
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5>
```

c) BST Insert

```
1 public boolean insert(int value) {
2     Node newNode = new Node(value);
3     if (root == null) {
4         root = newNode;
5         return true;
6     }
7     Node temp = root;
8     while (true) {
9         if (newNode.value == temp.value) {
10             return false;
11         }
12         if (newNode.value < temp.value) {
13             if (temp.left == null) {
14                 temp.left = newNode;
15                 return true;
16             }
17             temp = temp.left;
18         } else {
19             if (temp.right == null) {
20                 temp.right = newNode;
21                 return true;
22             }
23             temp = temp.right;
24         }
25     }
26 }
```

Berdasarkan analisis yang dilakukan, konsep Linked List dalam struktur data memberikan fleksibilitas dalam manipulasi data, terutama dalam operasi pencarian (find), pengurutan (sorting), dan penyalinan (copy). Dalam implementasi menggunakan Java, penggunaan Comparable dengan metode compareTo menjadi penting untuk menentukan aturan perbandingan antar elemen. Selain itu, penerapan static pada atribut atau metode memungkinkan akses yang lebih efisien tanpa perlu instansiasi objek. Dalam pemodelan, Class Diagram membantu menggambarkan relasi antar kelas dan keterkaitan data. Penggunaan Generic dalam Java juga menjadi solusi untuk membuat struktur data lebih fleksibel dan reusable tanpa bergantung pada tipe data tertentu. Dengan pemahaman yang baik terhadap konsep-konsep ini, implementasi struktur data Linked List dapat lebih optimal dalam berbagai kasus penggunaan.

```
1 package del.ifs23043;
2
3 public class App {
4     public static void main(String[] args) {
5         BinarySearchTree myTree = new BinarySearchTree();
6         myTree.insert(47);
7         myTree.insert(21);
8         myTree.insert(76);
9         myTree.insert(18);
10        myTree.insert(52);
11        myTree.insert(82);
12        myTree.insert(27);
13
14        System.out.println(myTree.root.left.right.value);
15    }
16 }
17
```

Program ini mengimplementasikan struktur **Binary Search Tree (BST)** dalam Java, di mana data disusun berdasarkan aturan bahwa nilai yang lebih kecil dari root akan ditempatkan di kiri, sedangkan nilai yang lebih besar berada di kanan. Dalam program ini, objek myTree dibuat dari kelas BinarySearchTree, kemudian beberapa angka dimasukkan menggunakan metode insert(). Saat angka-angka tersebut dimasukkan secara berurutan, pohon yang terbentuk memiliki 47 sebagai root, dengan 21 di kiri dan 76 di kanan. Selanjutnya, 18 ditempatkan di kiri 21, 27 di kanan 21, 52 di kiri 76, dan 82 di kanan 76. Ketika perintah `System.out.println(myTree.root.left.right.value);` dieksekusi, program akan mengakses nilai yang berada di posisi **root** → **left** → **right**, yang dalam struktur BST ini berarti nilai 27. Dengan demikian, program ini tidak hanya menunjukkan bagaimana BST bekerja dalam menyusun data, tetapi juga bagaimana elemen tertentu dapat diakses berdasarkan hierarki dalam struktur pohon.

```
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.854 s
[INFO] Finished at: 2025-03-18T14:05:32+07:00
[INFO] -----
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> java -cp target/ifs23043-alstrudat-p5-1.0-SNAPSHOT.jar de
l.ifs23043.App
27
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> 
```

d) BST Contains

```
1 public boolean contains(int value) {
2     if (root == null) {
3         return false;
4     }
5     Node temp = root;
6     while (temp != null) {
7         if (value < temp.value) {
8             temp = temp.left;
9         } else if (value > temp.value) {
10            temp = temp.right;
11        } else {
12            return true;
13        }
14    }
15    return false;
16 }
17 }
```

Metode `contains(int value)` pada program ini berfungsi untuk mengecek apakah suatu nilai terdapat dalam **Binary Search Tree (BST)**. Proses dimulai dengan memeriksa apakah `root` bernilai `null`, yang berarti pohon kosong dan langsung mengembalikan `false`. Jika pohon tidak kosong, variabel `temp` digunakan untuk menelusuri node, dimulai dari `root`. Perulangan `while (temp != null)` memastikan pencarian berlangsung selama node yang diperiksa tidak `null`. Jika nilai yang dicari lebih kecil dari nilai pada node saat ini, pencarian berlanjut ke **subtree kiri** dengan `temp = temp.left`. Sebaliknya, jika nilai lebih besar, pencarian berlanjut ke **subtree kanan** dengan `temp = temp.right`. Jika ditemukan nilai yang sama, metode langsung mengembalikan `true`, menandakan bahwa nilai tersebut ada dalam BST. Jika perulangan berakhir tanpa menemukan nilai, metode mengembalikan `false`, menandakan bahwa nilai tidak ditemukan dalam pohon. Dengan pendekatan ini, pencarian dalam BST dilakukan secara efisien dengan kompleksitas waktu **$O(\log n)$** pada BST yang seimbang, tetapi bisa mencapai **$O(n)$** jika pohon tidak seimbang.

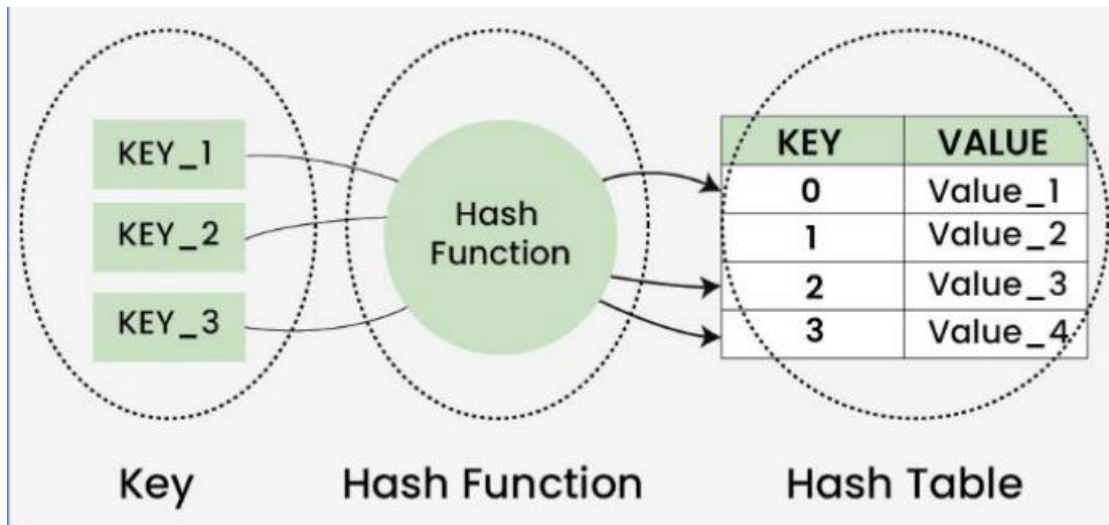
```
1 package del.ifs23043;
2 public class App {
3     public static void main(String[] args) {
4         BinarySearchTree myTree = new BinarySearchTree();
5
6         myTree.insert(47);
7         myTree.insert(21);
8         myTree.insert(76);
9         myTree.insert(18);
10        myTree.insert(52);
11        myTree.insert(82);
12        myTree.insert(27);
13
14        System.out.println(myTree.contains(27));
15        System.out.println(myTree.contains(17));
16    }
17 }
```

Program ini mengimplementasikan **Binary Search Tree (BST)** dalam Java dan menguji metode `contains()` untuk mengecek keberadaan nilai dalam pohon. Pertama, objek `myTree` dibuat dari kelas `BinarySearchTree`, lalu beberapa angka dimasukkan ke dalamnya menggunakan metode `insert()`, membentuk struktur BST dengan 47 sebagai root. Node 21 berada di kiri 47, sedangkan 76 di kanan. Selanjutnya, 18 berada di kiri 21, 27 di kanan 21, 52 di kiri 76, dan 82 di kanan 76. Setelah BST terbentuk, program memanggil `contains(27)` dan `contains(17)`. Metode `contains()` bekerja dengan membandingkan nilai yang dicari terhadap root dan menelusuri subtree kiri atau kanan sesuai aturan BST. Untuk `contains(27)`, program akan menemukan angka tersebut dalam subtree kiri 47, sehingga hasilnya `true`. Sebaliknya, `contains(17)` tidak ditemukan dalam pohon, sehingga metode mengembalikan `false`. Dengan demikian, program ini menunjukkan bagaimana BST menyimpan data dan melakukan pencarian secara efisien berdasarkan struktur hierarkinya.

```
-----
[INFO] Total time: 3.816 s
[INFO] Finished at: 2025-03-18T14:10:07:00
[INFO] -----
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> java -cp target/ifs23043-alstrudat-p5-1.0-SNAPSHOT.jar de
l.ifs23043.App
27
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> 
```

2. Hash Table

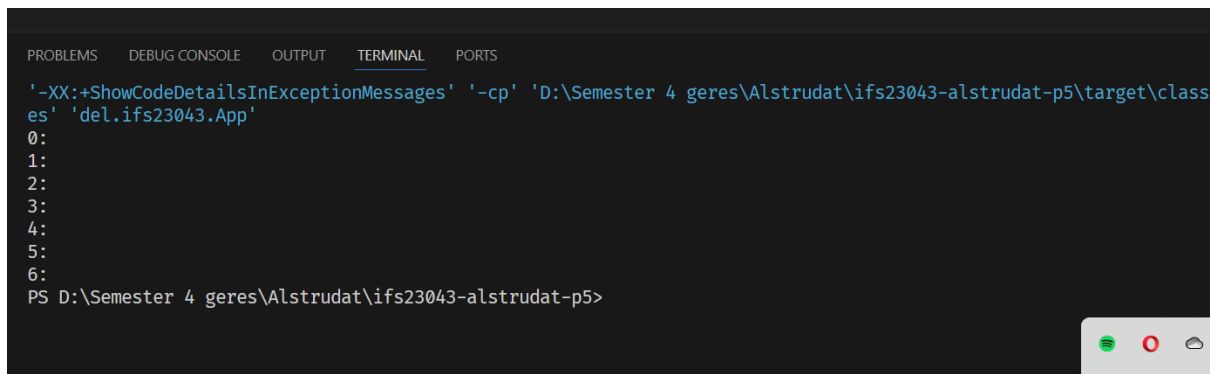
a) Intro



b) Constructor

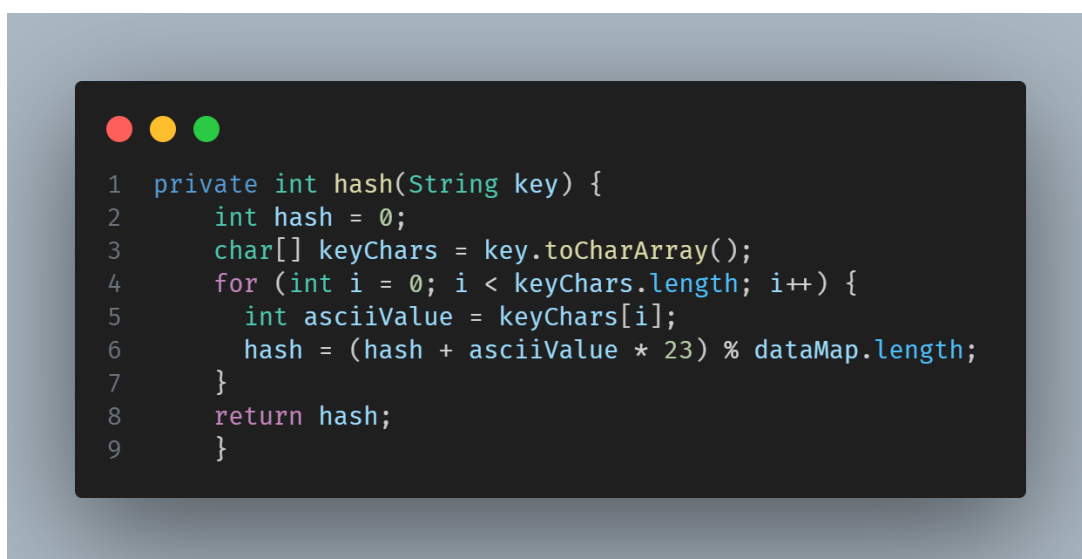
```
1 package del.ifs23043;
2
3 public class HashTable {
4     private int size = 7;
5     private Node[] dataMap;
6
7     class Node {
8         String key;
9         int value;
10        Node next;
11
12        Node(String key, int value) {
13            this.key = key;
14            this.value = value;
15        }
16    }
17    public HashTable() {
18        dataMap = new Node[size];
19    }
20    public void printTable() {
21        for (int i = 0; i < dataMap.length; i++) {
22            System.out.println(i + ":");
23            Node temp = dataMap[i];
24            while (temp != null) {
25                System.out.println(" {" + temp.key + " = " + temp.value +
26                " }");
27                temp = temp.next;
28            }
29        }
30    }
31 }
32
33
```

Program ini mengimplementasikan struktur **Hash Table** dalam Java menggunakan array dengan teknik **chaining (linked list)** untuk menangani **collision**. Kelas `HashTable` memiliki atribut `size` sebesar 7 dan array `dataMap` untuk menyimpan node yang berisi pasangan **key-value**. Setiap node direpresentasikan oleh kelas `Node`, yang memiliki atribut `key`, `value`, dan `next`, memungkinkan penanganan tabrakan hash dengan menyambungkan elemen baru dalam linked list. Konstruktor `HashTable()` menginisialisasi `dataMap` dengan ukuran yang telah ditentukan. Metode `printTable()` digunakan untuk mencetak isi tabel hash dengan menelusuri setiap indeks dalam array dan mencetak pasangan **key-value** jika ada, atau hanya menampilkan nomor indeks jika kosong. Namun, program ini masih belum memiliki metode untuk hashing, penyisipan (`insert`), pencarian (`get`), atau penghapusan (`delete`), sehingga fungsionalitasnya masih terbatas pada struktur dasar tanpa operasi utama hash table.



```
PROBLEMS  DEBUG CONSOLE  OUTPUT  TERMINAL  PORTS
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5\target\classes' 'del.ifs23043.App'
0:
1:
2:
3:
4:
5:
6:
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5>
```

c) Hash Method



```
1 private int hash(String key) {
2     int hash = 0;
3     char[] keyChars = key.toCharArray();
4     for (int i = 0; i < keyChars.length; i++) {
5         int asciiValue = keyChars[i];
6         hash = (hash + asciiValue * 23) % dataMap.length;
7     }
8     return hash;
9 }
```

Metode `hash(String key)` pada program ini berfungsi untuk menghitung indeks dalam tabel hash berdasarkan nilai string yang diberikan. Prosesnya dimulai dengan inisialisasi variabel `hash` ke 0, lalu string `key` diubah menjadi array karakter menggunakan `toCharArray()`, memungkinkan iterasi melalui setiap karakter dalam string. Dalam perulangan, nilai ASCII dari setiap karakter dikalikan dengan konstanta **23**, kemudian hasilnya dijumlahkan dengan nilai `hash` sebelumnya. Hasil akhir dari perhitungan ini diambil modulus terhadap panjang array `dataMap`, memastikan bahwa nilai `hash` selalu berada dalam rentang indeks yang valid dalam tabel hash. Pendekatan ini bertujuan untuk menyebarkan nilai hash secara lebih merata dalam tabel, mengurangi kemungkinan **collision**. Namun, metode ini masih sederhana dan tidak menggunakan teknik hashing yang lebih kompleks seperti **double hashing** atau **universal hashing**, yang dapat meningkatkan distribusi data dalam tabel hash.

d) Set

```
1 public void set(String key, int value) {
2     int index = hash(key);
3     Node newNode = new Node(key, value);
4     if (dataMap[index] == null) {
5         dataMap[index] = newNode;
6     } else {
7         Node temp = dataMap[index];
8         while (temp.next != null) {
9             temp = temp.next;
10        }
11        temp.next = newNode;
12    }
13 }
```

Metode `set(String key, int value)` dalam program ini digunakan untuk menyisipkan pasangan **key-value** ke dalam **Hash Table** menggunakan teknik **chaining (linked list)** untuk menangani **collision**. Pertama, indeks dihitung menggunakan metode `hash(key)`, yang memastikan bahwa setiap **key** dikonversi ke indeks yang sesuai dalam array `dataMap`. Setelah indeks diperoleh, program membuat node baru dengan nilai **key-value**. Jika tidak ada entri pada indeks tersebut (`dataMap[index] == null`), maka node baru langsung disimpan di indeks tersebut. Namun, jika sudah ada node, artinya terjadi **collision**, sehingga program akan menelusuri linked list pada indeks tersebut menggunakan perulangan `while (temp.next != null)`, hingga menemukan node terakhir, lalu menambahkan node baru di akhir list dengan `temp.next = newNode`. Dengan metode ini, hash table dapat menyimpan beberapa elemen pada indeks yang sama tanpa menimpa data yang sudah ada. Namun, metode ini belum menangani kasus ketika **key** yang sama dimasukkan kembali, sehingga dapat terjadi duplikasi entri dalam tabel hash.

```
1 package del.ifs23043;
2 public class App {
3     public static void main(String[] args) {
4         HashTable myHashTable = new HashTable();
5
6         myHashTable.set("nails", 100);
7         myHashTable.set("tile", 100);
8         myHashTable.set("lumber", 100);
9
10        myHashTable.set("bolts", 200);
11        myHashTable.set("screws", 140);
12
13        myHashTable.printTable();
14    }
15 }
16
```

Program ini menguji implementasi **Hash Table** dengan metode **chaining (linked list)** untuk menangani **collision** dalam penyimpanan data. Di dalam `main()`, objek `myHashTable` dibuat dari kelas `HashTable`, lalu beberapa pasangan **key-value** dimasukkan menggunakan metode `set()`. Setiap **key** seperti "nails", "tile", dan "lumber" diberikan nilai 100, sedangkan "bolts" bernilai 200 dan "screws" bernilai 140. Metode `set()` pertama-tama menghitung indeks dalam array `dataMap` menggunakan fungsi `hash()`, lalu menambahkan node baru pada indeks tersebut. Jika terjadi **collision**, node baru akan ditambahkan ke dalam **linked list** pada indeks yang sama. Setelah semua data dimasukkan, metode `printTable()` dipanggil untuk mencetak isi tabel hash, menampilkan setiap indeks beserta daftar pasangan **key-value** yang tersimpan di dalamnya. Program ini menunjukkan bagaimana hash table menyebarkan data berdasarkan nilai hash dari **key**, tetapi masih memiliki keterbatasan karena belum menangani pembaruan nilai jika **key** yang sama dimasukkan kembali, yang dapat menyebabkan duplikasi entri.

```
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.057 s
[INFO] Finished at: 2025-03-18T14:30:31+07:00
[INFO] -----
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> java -cp target/ifs23043-alstrudat-p5-1.0-SNAPSHOT.jar de
l.ifs23043.App
0:
1:
2:
3:
4: {screws= 140}
5: {bolts= 200}
6: {nails= 100}
   {tile= 100}
   {lumber= 100}
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> 
```

e) Get

```
1 public int get(String key) {
2     int index = hash(key);
3     Node temp = dataMap[index];
4     while (temp != null) {
5         if (temp.key.equals(key)) {
6             return temp.value;
7         }
8         temp = temp.next;
9     }
10    return 0;
11 }
```

Metode `get(String key)` dalam program ini berfungsi untuk mengambil nilai (value) berdasarkan **key** dalam **Hash Table**. Pertama, metode ini menghitung indeks menggunakan `hash(key)`, lalu mengakses indeks tersebut dalam array `dataMap`. Jika terdapat node (`temp != null`), maka program akan menelusuri linked list pada indeks tersebut menggunakan perulangan `while`, mengecek apakah **key** pada node saat ini (`temp.key.equals(key)`) sesuai dengan **key** yang dicari. Jika ditemukan, nilai (value) dari node tersebut dikembalikan. Jika tidak, program akan terus berpindah ke node berikutnya (`temp = temp.next`) hingga akhir linked list. Jika **key** tidak ditemukan dalam tabel hash, metode ini mengembalikan 0 sebagai nilai default. Metode ini memastikan bahwa pencarian tetap efisien dalam kondisi normal, tetapi jika

terlalu banyak **collision**, maka pencarian bisa menjadi lebih lambat karena harus menelusuri banyak node dalam linked list. Selain itu, pengembalian 0 sebagai nilai default bisa menjadi masalah jika nilai 0 adalah nilai yang valid dalam tabel, sehingga akan lebih baik jika menggunakan null atau nilai khusus untuk membedakan antara **key** yang tidak ditemukan dan nilai yang valid.

```
1 package del.ifs23043;
2 public class App {
3     public static void main(String[] args) {
4         HashTable myHashTable = new HashTable();
5
6         myHashTable.set("nails", 100);
7         myHashTable.set("tile", 59);
8         myHashTable.set("lumber", 77);
9
10        System.out.println(myHashTable.get("lumber"));
11        System.out.println(myHashTable.get("bolts"));
12    }
13 }
```

Program ini menguji fungsionalitas **Hash Table**, terutama metode `set()` untuk memasukkan data dan `get()` untuk mengambil nilai berdasarkan **key**. Objek `myHashTable` dibuat dari kelas `HashTable`, lalu beberapa pasangan **key-value** dimasukkan: "nails" dengan nilai 100, "tile" dengan nilai 59, dan "lumber" dengan nilai 77. Metode `set()` menghitung indeks hash dari **key**, lalu menyimpan nilai dalam array `dataMap`, menggunakan **chaining (linked list)** jika terjadi **collision**. Setelah data dimasukkan, program memanggil `get("lumber")`, yang akan mencari node dengan **key** "lumber" pada indeks yang sesuai dalam tabel hash, lalu mengembalikan nilai 77. Namun, ketika `get("bolts")` dipanggil, **key** "bolts" belum pernah dimasukkan, sehingga metode akan menelusuri indeks yang dihitung berdasarkan hash dan tidak menemukan node dengan **key** tersebut, sehingga kemungkinan besar akan mengembalikan null atau nilai default. Program ini

menunjukkan bagaimana **Hash Table** bekerja dalam menyimpan dan mengambil data, tetapi masih memiliki kekurangan karena tidak menangani pembaruan nilai jika **key** yang sama dimasukkan kembali, serta belum ada mekanisme untuk menangani penghapusan data.

```
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.993 s
[INFO] Finished at: 2025-03-18T14:37:58+07:00
[INFO] -----
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> java -cp target/ifs23043-alstrudat-p5-l.ifs23043.App
77
0
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> 
```

f) Keys

```
1 public ArrayList keys() {
2     ArrayList<String> allKeys = new ArrayList<>();
3     for (Node temp : dataMap) {
4         while (temp != null) {
5             allKeys.add(temp.key);
6             temp = temp.next;
7         }
8     }
9     return allKeys;
10 }
```

Metode `keys()` dalam program ini digunakan untuk mengembalikan semua **key** yang tersimpan dalam **Hash Table** dalam bentuk `ArrayList<String>`. Metode ini pertama-tama membuat objek `allKeys`, sebuah **ArrayList** yang akan menyimpan semua **key** yang ditemukan. Kemudian, ia melakukan iterasi melalui setiap indeks dalam array `dataMap` menggunakan perulangan `for-each`, di mana setiap indeks berisi sebuah node atau `null` jika kosong. Jika terdapat node (`temp != null`), program akan masuk ke dalam perulangan `while` untuk menelusuri seluruh **linked list** di indeks tersebut (jika terjadi **collision**). Setiap **key** yang ditemukan ditambahkan ke dalam `allKeys` menggunakan `add()`, dan iterasi dilanjutkan hingga semua node dalam tabel hash telah diperiksa. Terakhir, metode ini mengembalikan `allKeys`, yang berisi daftar lengkap semua **key** yang tersimpan. Pendekatan ini memastikan bahwa semua **key** dalam tabel hash dapat diakses, tetapi efisiensinya tergantung pada jumlah **collision** yang terjadi—jika

banyak **key** berada di indeks yang sama, iterasi akan lebih panjang. Selain itu, metode ini bisa ditingkatkan dengan menggunakan parameterisasi generik (`ArrayList<String> allKeys = new ArrayList<>();`) agar lebih sesuai dengan standar **Java Generics**.

```
-----  
[INFO] BUILD SUCCESS  
[INFO] -----  
[INFO] Total time: 4.254 s  
[INFO] Finished at: 2025-03-18T14:41:06+07:00  
[INFO] -----  
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> java -cp target/ifs23043-alstrudat-p5-1.0-SNAPSHOT.jar de  
l.ifs23043.App  
[screws, bolts, nails, tile, lumber]  
PS D:\Semester 4 geres\Alstrudat\ifs23043-alstrudat-p5> █
```

3. Studi Kasus Binary Search Tree

- Hapus Node Tree Buatlah method baru untuk menghapus node pada tree. Posisi node yang dihapus akan digantikan oleh child atau sub-tree jika tersedia.
- Jumlah Node Buatlah method baru untuk menghitung jumlah keseluruhan node pada tree.
- Jumlah Daun Buatlah method baru untuk menghitung jumlah daun pada tree.

4. Studi Kasus Hash Table

- Temukan Duplikasi Diberikan sebuah array temukanlah nilai duplikatnya dengan memanfaatkan Hash Table. Contoh [3, 3, 4, 2, 7, 8, 1, 2] hasilnya adalah [3, 2].
- Karakter Pertama yang Tidak Berulang Diberikan sebuah string temukanlah karakter pertama yang tidak berukang dengan memanfaatkan Hash Table. Contoh "abdullah" hasilnya adalah "b" karena karakter pertama yang tidak berulang.

5. Menghitung Big O

- Big O : Binary Search Tree
- Big O : Hash Table